

T13 — Classi Astratte ed Interfacce

Contratti Comportamentali, Metodi Astratti, Default e Static Methods, Builder Pattern e Lambda

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo a fondo i meccanismi di astrazione pura di Java: le **Classi Astratte**, le **Interfacce**, il design pattern **Template Method** e le interfacce cardine della libreria standard (`Comparable`, `Iterable`, `Iterator`) ([T13 - Abstract Classes and Interfaces.pdf](#)). Nel codice di [Lezione13/MavenDate](#), il docente compie un’evoluzione straordinaria introducendo: 1. La classe statica annidata `Date.Builder` per la costruzione controllata degli oggetti. 2. L’interfaccia funzionale `FormattedDateConverter`. 3. L’uso congiunto in `MainDate.java` di **Classi Anonime** (`new Time(){ ... }`) e **Espressioni Lambda** (`d -> new AmericanDate(...)`).

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T13)

A. Classi Astratte e Template Method Pattern (Slide 1–10)

- **Metodo Astratto:**

- Un metodo dichiarato con la parola chiave `abstract` e **privo di corpo** (termina con il punto e virgola `;` anziché con le parentesi graffe `{ ... }`):

```
1 public abstract String prettyPrint();
```

- Definisce una “firma obbligatoria”: impone alle sottoclassi concrete il compito di implementarne la logica.

- **Classe Astratta** (`abstract class`):

- Se una classe contiene anche un solo metodo astratto, **deve obbligatoriamente essere dichiarata `abstract`**.
- Attenzione: **Regola aurea d’esame**: Una classe astratta **non può MAI essere istanziata direttamente** (`new FormattedDate(...)` genera un **Compile-Time Error!**).
- Può contenere costruttori (invocabili dalle sottoclassi tramite `super(...)`), campi di istanza (`protected/private`), metodi concreti e metodi astratti.
- Una sottoclasse che estende una classe astratta deve:
 - * O implementare **tutti** i metodi astratti ereditati.
 - * Oppure essere dichiarata essa stessa `abstract`.

- **Il Template Method Pattern:**

- Pattern comportamentale in cui la superclasse definisce lo scheletro immutabile di un algoritmo all’interno di un metodo concreto marcato **`final`** (il template), mentre delega i singoli passi variabili o dipendenti dal contesto a metodi `abstract` implementati dalle sottoclassi.

- Nel nostro progetto: `printFormat()` e `getMonthAsString()` SONO `final` in `FormattedDate`, mentre `prettyPrint()` è `abstract`.

B. Le Interfacce in Java (`interface`) (Slide 11–17)

• Definizione e Filosofia:

- Un’interfaccia è un **contratto puro di comportamento**. Rappresenta ciò che una classe *sa fare* (*can-do*), non la sua identità ontologica.
- **Membri di un’interfaccia standard:**
 - * **Campi:** sono implicitamente ed esclusivamente `public static final` (costanti di classe). Non possono esistere variabili d’istanza o stato mutabile.
 - * **Metodi:** sono implicitamente `public abstract` (non serve specificarlo).

• Ereditarietà Multipla di Tipo (`implements`):

- Una classe può estendere al massimo una sola superclasse, ma può implementare **un numero arbitrario di interfacce separate da virgola**:

```
1 public class TimeStamp extends Date implements Time, Serializable, Cloneable
```

- Questo realizza in Java l’ereditarietà multipla dei tipi senza incorrere nei problemi di ambiguità di memoria del C++.

• Ereditarietà tra Interfacce:

- Un’interfaccia può estendere altre interfacce tramite `extends` (anche multiple contemporaneamente: `interface C extends A, B`).

C. Confronto Sistematico: Classe Astratta vs Interfaccia

Proprietà	Classe Astratta (<code>abstract class</code>)	Interfaccia (<code>interface</code>)
Istanziabilità	[NO] No (<code>new</code> vietato)	[NO] No (<code>new</code> vietato)
Stato d’istanza (campi)	[OK] Sì (qualsiasi visibilità)	[NO] No (solo costanti <code>public static final</code>)
Costruttori	[OK] Sì (invocabili con <code>super(...)</code>)	[NO] No
Ereditarietà	Singola (<code>extends</code> una sola classe)	Multipla (<code>implements N</code> interfacce)
Relazione concettuale	Identità ontologica forte (<i>IS-A</i>)	Capacità / Contratto comportamentale (<i>CAN-DO</i>)

D. Interfacce Standard Fondamentali della JDK (Slide 18–22)

1. `java.lang.Comparable<T>`:

- Definisce il metodo `int compareTo(T other)`.
- Permette agli algoritmi di ordinamento (`Arrays.sort()`, `Collections.sort()`) e agli insiemi ordinati (`TreeSet`, `TreeMap`) di ordinare gli oggetti secondo il loro “ordine naturale”.

2. `java.lang.Iterable<T>` e `java.util.Iterator<T>`:

- **`Iterable<T>`**: espone il metodo `Iterator<T> iterator()`.
- **Regola cruciale**: Qualsiasi classe che implementi `Iterable` può essere utilizzata come sorgente nel **ciclo For-Each** (`for (T elem : collection)`)!
- **`Iterator<T>`**: l'oggetto cursore che scorre la sequenza:
 - `boolean hasNext()`: verifica se vi sono ulteriori elementi.
 - `T next()`: restituisce il prossimo elemento avanzando il cursore.
 - `default void remove()`: rimuove l'ultimo elemento restituito.

0.0.2 2. Codice: Evoluzione del Codice: [Lezione13/MavenDate](#)

1. Il Design Pattern Builder con Classe Statica Annidata (`Date.java`) All'interno di `Date.java`, il docente definisce una **Static Nested Class** per costruire date in modo protetto:

```

1 public class Date implements Comparable {
2     ...
3     public static class Builder {
4         private final int year;
5
6         public Builder(int year) {
7             if (year > 0)
8                 this.year = year;
9             else
10                this.year = 1970; // Anno di default sicuro
11        }
12
13        public Date build(int day, int month) {
14            // Se i parametri sono illegali, ricade sulla data sicura 1/1/year!
15            if (month < 1 || month > 12 || day < 1 || day > daysPerMonth(month))
16                return new Date(1, 1, year);
17            return new Date(day, month, year);
18        }
19    }
20 }
```

Vantaggio del Builder: Incapsula e pre-configura l'anno; se l'utente fornisce parametri non validi (`month = -1`), il costruttore non fallisce a video ma restituisce una data valida di fallback (`01/01/year`).

2. L'Interfaccia Funzionale: `FormattedDateConverter.java`

```

1 package it.oop.core;
2
3 public interface FormattedDateConverter {
4     FormattedDate convert(FormattedDate date);
5 }
```

Questa interfaccia dichiara **un solo metodo astratto** (SAM: *Single Abstract Method*). In Java è a tutti gli effetti un' **Interfaccia Funzionale**, target ideale per le espressioni Lambda!

3. Sintesi Pratica in `MainDate.java` Questo file è una miniera di concetti d'esame avanzati:

```

1 package it.oop.ui;
2
3 import it.oop.core.*;
4 import it.oop.core.Date;
```

```

5
6 public class MainDate {
7     public static void main(String[] args) {
8         // 1. Uso della Static Nested Class Builder:
9         Date.Builder dateBuilder = new Date.Builder(2025);
10        Date d1 = dateBuilder.build(10, 9); // Giorno 10, Mese 9 -> Valida!
11        Date d2 = dateBuilder.build(10, -1); // Mese -1 illegale -> Fallback su 1/1/2025!
12        System.out.println(d1.toString()); // y2025m9d10
13        System.out.println(d2.toString()); // y2025m1d1
14
15        // 2. Classe Anonima (Anonymous Inner Class):
16        // Implementa al volo l'interfaccia Time senza creare un file .java separato!
17        Time init = new Time() {
18            @Override
19            public int getHours() { return 0; }
20            @Override
21            public int getMinutes() { return 0; }
22            @Override
23            public int getSeconds() { return 0; }
24            @Override
25            public String toString() {
26                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());
27            }
28        };
29        System.out.println(init.toString()); // Stampa 00:00:00
30
31        // 3. Espressione Lambda (Sintassi compatta per Interfaccia Funzionale):
32        FormattedDateConverter toAmerican =
33            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());
34
35        // Test di conversione polimorfica:
36        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof
37            AmericanDate);
38        // Stampa TRUE!
39    }
}

```

0.0.3 3. Test: Output di Esecuzione Verificato con Maven

Eseguendo `mvn exec:java -Dexec.mainClass="it.oop.ui.MainDate"`:

```

1 y2025m9d10
2 y2025m1d1
3 00:00:00
4 true

```

Nota di Studio

Con la **Lezione 13** abbiamo chiuso il cerchio su Classi Astratte, Interfacce, Template Method e abbiamo anticipato le classi annidate e le lambda.

Il prossimo blocco è la **Lezione 14 / Slide T14: *Nested and Anonymous Classes***: - Tassonomia completa delle classi interne: **Static Nested Classes**, **Inner Member Classes** (non statiche), **Local Classes** e **Anonymous Classes**. - Accesso allo stato della classe contenitore (*Enclosing instance*) e la sintassi speciale `EnclosingClass.this`. - Cattura delle variabili locali nelle classi locali/anonime e il vincolo di essere **effectively final**. - Evoluzione del codice in Lezione14: consolidamento delle classi interne e preparativi per i Generics.

Dammi conferma per aprire la Lezione 14 / T14 e continuare la revisione!